

Prompt de implementação — Semana 1

Simulador de posicionamento de satélites (satsim) · Cards S1-01 a S1-08 · Fundação, núcleo kepleriano e primeiro protótipo visual (GUI)

Como usar (para você, humano)

1. Salve este PDF na raiz do repositório (ou em docs/). Opcional: coloque os dois PDFs teóricos (Semana 3, decaimento em LEO; Semana 4, órbita heliossíncrona) em docs/referencias/.
2. Abra o Claude Code no VSCode, na pasta do repositório, e envie: “Leia o arquivo PROMPT_SEMANA1_claude_code.pdf inteiro e siga as instruções. Comece pelo card S1-01.”
3. O Claude Code executa **um card** e para com um relatório. Confira o código (`git show`), rode os comandos sugeridos e responda “aprovado” para seguir, ou peça ajustes.
4. O critério “verde no CI” do S1-01 só pode ser confirmado por você: faça o `git push` e veja a aba Actions.
5. No S1-08 você avalia o **visual**: abra `demo/index.html` (duplo clique) e diga ao Claude Code o que quer ajustar.

A partir daqui, o texto é dirigido ao Claude Code.

1. Visão do produto — requisito central de GUI

O produto final não é uma biblioteca de scripts: é um simulador com interface gráfica (GUI). Uma pessoa qualquer, sem conhecimento técnico, deve abrir o simulador e **ver os satélites se movendo sobre o globo terrestre**, com **forte impacto visual**. Na **mesma tela**, ao mesmo tempo e **sincronizados no tempo**:

- **(A) Globo terrestre em 3D**, com a Terra texturizada e os satélites em movimento em suas órbitas.
- **(B) Mapa em 2D com o traço no solo** (ground track) dos mesmos satélites: subponto atual, trecho já percorrido e trecho futuro.
- **(C) Controles** de tempo: reproduzir/pausar, velocidade, linha do tempo e relógio UTC.

Toda decisão de arquitetura, de formato de dados e de código, desde o primeiro card, deve servir a este objetivo. O simulador numérico (Python) existe para alimentar essa GUI.

Requisitos de GUI (R1-R10)

#	Requisito
R1	Globo 3D com Terra texturizada (textura pública), fundo espacial com estrelas, brilho atmosférico e câmera controlável (arrastar para girar, roda para zoom).
R2	Satélites em movimento contínuo (interpolação fluida, ≥ 30 fps), como marcadores luminosos com nome, cor própria, trilha recente e órbita prevista .
R3	Mapa 2D (projeção equirretangular, mesma textura da Terra) na mesma tela: subponto atual, traço no solo passado (linha cheia) e futuro (tracejada) , quebra correta no antimeridiano ($\pm 180^\circ$, sem linha atravessando o mapa) e grade de latitude/longitude.
R4	Sincronia : uma única fonte de tempo (<code>simTime</code>) comanda o 3D e o 2D. Arrastar a linha do tempo atualiza os dois painéis.
R5	Controles : play/pause, velocidade (1x, 10x, 60x, 300x, 1000x), linha do tempo, relógio UTC visível, botão “agora/início”, mostrar/ocultar cada satélite.
R6	Estação de solo (São Bento do Sapucaí, $\approx -22,69^\circ$, $-45,73^\circ$) marcada nos dois painéis; a partir da Semana 4, destaque visual quando o satélite estiver visível.
R7	Escala de 1 → 6 → 18 satélites sem alterar o código do viewer: os satélites entram por dados/configuração.
R8	Impacto visual : tema escuro, cores vivas, animação fluida, tipografia limpa; layout responsivo (3D e 2D lado a lado em tela larga, empilhados em tela estreita).

#	Requisito
R9	Uso simples: abre com um comando ou duplo clique, sem configuração; legenda e dicas na tela.
R10	Offline após clonar: bibliotecas JS e texturas versionadas no repositório (static/vendor, static/textures), com créditos e licenças em docs/creditos.md.

Layout-alvo da tela

+-----+ satsim UTC 2026-09-29 12:00:00 [>][] [1x 10x 60x 300x 1000x] [agora] +-----+		
	GLOBO 3D	MAPA 2D – traço no solo
	satélites em movimento	subponto + passado/futuro
	órbitas, trilhas, estação	estação de solo marcada
		+-----+
		Legenda · satélites · informações
+-----+		+-----+
	-----o-----	linha do tempo
+-----+		+-----+

Princípios de arquitetura decorrentes

- **A física fica em Python; o front-end só desenha.** Nenhuma equação orbital em JavaScript (apenas interpolação temporal e a projeção/rotação para desenhar).
- **Contrato de dados da “cena” (JSON)** entre Python e viewer, definido no ADR (S1-02) e implementado no S1-08: trajetórias amostradas no tempo (posição ECI e latitude/longitude/altitude), estações e, mais tarde, passagens.
- **Stack recomendada** (o ADR confirma ou justifica a troca): viewer HTML/JS estático e autocontido — **globe.gl** (bundle UMD, baseado em three.js) para o 3D e **canvas 2D** desenhando a mesma textura equiretangular para o mapa — alimentado por JSON gerado em Python. Na Semana 6, um serviço Python (Streamlit ou FastAPI) hospeda o viewer no deploy. Alternativas a comparar no ADR: CesiumJS, Plotly, PyVista/VTK (desktop), Streamlit puro.
- **Restrição de file:///:** o viewer de demonstração deve abrir com duplo clique. Navegadores bloqueiam módulos ES e texturas carregadas de file:///; portanto use scripts clássicos (UMD) e incorpore texturas e JSON da cena como data URI / `<script>` embutido.

O que isso muda na Semana 1

- **S1-02 (ADR):** deve conter uma seção de GUI completa (requisitos, layout, contrato JSON, decisão de stack com alternativas).
- **S1-08 (novo):** primeiro protótipo visual — globo 3D + traço 2D animados na mesma tela, com o propagador kepleriano. A GUI completa (ISS real, passagens, 6 e 18 satélites, deploy) vem nas semanas seguintes, sempre reaproveitando este viewer.
- **Todos os cards:** funções vetorizadas e trajetórias em arrays (N, 3), porque é isso que a GUI consome.
- **Fora do escopo da Semana 1:** passagens, visibilidade, eclipse, SGP4/TLE, serviço web e deploy.

2. Papel e contexto

Você é um(a) engenheiro(a) Python sênior implementando, no repositório atual, o **satsim**: um simulador de posicionamento de satélites em código Python, modular e fácil de adaptar para mais satélites. O projeto dura 7 semanas:

- **Fase 1 (semanas 1-4):** um satélite (a ISS) com posição e horários de visibilidade sobre São Bento do Sapucaí (SP), **visualizado na GUI** (globo 3D + traço 2D).
- **Fase 2 (semana 5):** generalizar para 6 satélites (constelação heliossíncrona 91/6, um plano), também na GUI.
- **Fase 3 (semanas 6-7, se houver tempo):** aplicação web com deploy e 18 satélites, com polimento visual.

Esta é a **Semana 1 — fundação e núcleo kepleriano**: oito cards (S1-01 a S1-08). Ao final teremos repositório com CI, ADR de arquitetura (incluindo a GUI), constantes, conversão elementos ↔ estado, propagador kepleriano vetorizado, módulo de tempo, uma base de números de referência (golden tests) e um **primeiro protótipo visual** (globo 3D + traço no solo 2D animados).

Fonte de verdade numérica. Se docs/referencias/*.pdf existir, consulte-os. Caso contrário, os valores escritos neste prompt (principalmente no Apêndice A) são a fonte de verdade. **Nunca invente números de referência**: se precisar de um valor que não está aqui, pergunte.

Regra central de arquitetura. Todo propagador implementa `propagate(t) → (r, v)` vetorizado. Adicionar satélites deve ser apenas instanciar mais objetos `Satellite`; nenhum código de análise pode assumir um único satélite. E o resultado de toda propagação deve poder ser consumido pela GUI (seção 1).

3. Protocolo de trabalho (inegociável)

1. **Um card por vez**, na ordem S1-01 → S1-08. Não antecipe código de cards futuros, nem refatore além do escopo do card atual.
2. Ao concluir os passos de um card, rode `ruff check .`, `ruff format --check .` e `pytest` (quando já existirem) e confira **cada** critério de aceite individualmente.
3. **PARE.** Depois do relatório, não escreva mais código. Aguarde minha aprovação explícita (“aprovado” ou “próximo”). Se eu pedir ajustes, corrija dentro do mesmo card, refaça o relatório e pare de novo.
4. **Git:** trabalhe na branch `semana-1` (crie no início do S1-01). Faça **um commit por card** ao final dele, antes de parar, com a mensagem S1-0X: <título do card>, para que eu revise com `git show`. Ajustes de revisão viram commits extras S1-0X: ajustes de revisão. **Nunca** faça push, merge, rebase nem force push — isso é comigo.
5. **Ambiguidade:** se algo do card for ambíguo, ou se um critério de aceite não puder ser cumprido, não improvise em silêncio. Escolha a opção mais simples, declare-a no relatório e faça no máximo 1-2 perguntas objetivas.
6. **Dependências:** use apenas as listadas neste prompt. Se achar necessária outra, avise no relatório antes de adicioná-la.
7. **Honestidade nos testes:** nunca afrouxe uma tolerância nem apague um teste para “passar”. Se um teste falhar por causa de um número deste prompt, reporte a discrepância; não force o resultado.

Formato fixo do relatório de encerramento de cada card

1. **Resumo** em 3-5 linhas.
2. **Arquivos** criados e alterados (lista).
3. **Comandos rodados** e resultado (cole a saída resumida de `pytest` e `ruff`).
4. **Checklist dos critérios de aceite**, item a item, com [OK] ou [FALHA] e a evidência (teste, comando ou valor obtido).
5. **Como verificar manualmente:** comandos exatos que eu posso copiar e colar.
6. **Decisões, desvios e dúvidas em aberto.**
7. **Commit** feito (hash curto + mensagem).
8. Linha final exata: `CARD S1-0X CONCLUÍDO – aguardando sua aprovação para iniciar o S1-0Y.`

4. Convenções técnicas (valem para todos os cards)

- **Python ≥ 3.11**, layout `src/satsim`. Runtime: `numpy`, `scipy`. Dev: `pytest`, `pytest-cov`, `ruff`, `skyfield` (Skyfield é usado **só** para validação nos testes, nunca no runtime).
- **Unidades SI:** metros, segundos, kg, m/s. **Ângulos em radianos** internamente; graus só em entrada/saída e em nomes de teste com sufixo `_deg`.
- **Tempo:** UTC com `datetime *tz-aware*`. Nas funções de propagação, `t` = segundos (float) desde a época (epoch) do propagador.

- **Quadros:** ECI = equatorial inercial ($x \rightarrow$ equinócio vernal, $z \rightarrow$ eixo polar). ECEF = $R_z(\Theta_G)$ aplicado ao ECI. Simplificação assumida e documentada: sem precessão/nutação; $UT1 \approx UTC$.
- **Notação dos documentos** (diferente da literatura usual): ψ = anomalia média (o “M” da literatura); ℓ = momento angular específico $r \times v$ (o “h” da literatura); h = **altitude**; $\argp$ = argumento do perigeu (ω); $\text{raan} = \Omega$; ν = anomalia verdadeira; $u = \argp + \nu$ = argumento de latitude; $\text{OMEGA_EARTH} = \omega_\oplus$ (rotação da Terra).
- **Vetorização:** funções numéricas aceitam escalar ou `np.ndarray`. Shapes: tempos $(N,)$; posições e velocidades $(N, 3)$; constelação $(P, N, 3)$.
- **Estilo:** identificadores em inglês; docstrings, comentários, README e ADR em português; type hints; cada docstring cita a equação de origem quando houver (ex.: “Sem4 eq. 23”); sem estado global mutável; sem `print` em código de biblioteca (use `logging`).
- **GUI:** a física é calculada em Python e exportada como arrays amostrados no tempo (contrato “cena” do ADR); o viewer apenas desenha. Assets do viewer (JS, texturas) ficam versionados e com créditos em `docs/creditos.md`.
- **Testes:** `pytest`; tolerâncias explícitas (`pytest.approx`, `np.testing.assert_allclose`); todo teste que usa um número de referência cita a fonte em comentário (documento, seção, tabela ou equação).

5. Estrutura-alvo do repositório

A Semana 1 cria apenas os itens marcados com ●. O restante é o destino final, para você entender o encaixe.

```
satsim/                                (raiz do repositório)
├── pyproject.toml                      ●
├── README.md                          ●
├── .gitignore                         ● (inclui demo/)
├── .github/workflows/ci.yml           ●
├── scripts/build_demo.py              ● (gera demo/index.html)
├── demo/index.html                    ● (GERADO; abre com duplo clique)
├── docs/
│   ├── adr/0001-arquitetura.md        ●
│   ├── creditos.md                    ● (texturas e bibliotecas JS: fonte e licença)
│   └── referencias/                   ● (PDFs teóricos, opcional)
├── src/satsim/
│   ├── __init__.py                    ●
│   ├── constants.py                  ●
│   ├── astrotime.py                   ● (UTC/JD/GMST – ver nota abaixo)
│   ├── elements.py                   ●
│   └── propagators/
│       ├── __init__.py                ●
│       ├── base.py                    ● (ABC Propagator)
│       ├── kepler.py                  ●
│       ├── mean_j2.py · cowell.py · sgp4_ref.py (semanas 2–3)
│       ├── frames.py · geodesy.py · sun.py · observer.py · visibility.py (semanas 2–4)
│       ├── orbit_design.py · satellite.py · constellation.py · coverage.py (semanas 2–5)
│       └── app/                       ● (GUI)
│           ├── __init__.py · scene_export.py ● (Python → JSON da cena)
│           ├── static/ viewer.html · viewer.js · viewer.css ·
│           │   │   │   vendor/ (globe.gl UMD) · textures/ (Terra) ●
│           └── (serviço web Streamlit/FastAPI: semana 6)
├── tests/
│   ├── test_smoke.py · test_constants.py · test_elements.py ●
│   ├── test_kepler_propagator.py · test_astrotime.py        ●
│   ├── test_scene_export.py                                  ●
│   └── golden/reference_values.py · test_golden_consistency.py ●
```

Nota sobre o nome `astrotime.py`. O plano original chamava este módulo de `time.py`. Use `astrotime.py` para evitar confusão com o módulo `time` da biblioteca padrão. Registre isso no ADR (S1-02).

6. Cards da Semana 1

CARD S1-01 · Setup do projeto

2 pts

Objetivo. Criar a base do repositório: empacotamento, testes, lint e integração contínua.

Passos

1. Rode `git status` e liste os arquivos existentes. **Não sobrescreva** README, LICENSE ou .gitignore já presentes: mescle e mostre o que mudou.
2. Crie a branch `semana-1`.
3. Crie `pyproject.toml` (setuptools ou hatchling, à sua escolha; declare a escolha no relatório): nome `satsim`, versão `0.1.0`, `requires-python = ">=3.11"`, dependências `numpy>=1.26` e `scipy>=1.11`; extra dev com `pytest>=8`, `pytest-cov`, `ruff` e `skyfield`.
4. No `pyproject.toml`, configure `[tool.pytest.ini_options]` (`testpaths = ["tests"]`) e `[tool.ruff]` (`line-length = 100`, `target-version = "py311"`, `lint select = ["E", "F", "I", "B", "UP", "NPY"]`).
5. Crie `src/satsim/__init__.py` com docstring e `__version__ = "0.1.0"`, e `tests/test_smoke.py` que importa `satsim` e confere `__version__`.
6. Crie `.gitignore` para Python (`.venv/`, `__pycache__/`, `.pytest_cache/`, `.ruff_cache/`, `*.egg-info/`, `.coverage`, `htmlcov/`, `dist/`, `build/`, `.ipynb_checkpoints/`, `data/cache/`).
7. Crie README.md em português: o que é o projeto, instalação (`python -m venv .venv`, ativar, `pip install -e ".[dev]"`), comandos (`pytest`, `ruff check .`, `ruff format .`), estrutura resumida e o roadmap de 7 semanas em uma linha por semana.
8. Crie `.github/workflows/ci.yml`: dispare em push e pull_request; matriz Python 3.11 e 3.12; passos: checkout, setup-python, `pip install -e ".[dev]"`, `ruff check .`, `ruff format --check .`, `pytest`.
9. Crie um venv limpo, instale, rode `ruff` e `pytest` localmente. Valide a sintaxe do YAML com `python -c "import yaml; yaml.safe_load(open(...))"` (instale `pyyaml` apenas temporariamente, sem adicioná-lo ao projeto).

Critérios de aceite

- ☐ `pip install -e ".[dev]"` funciona em venv limpo.
- ☐ `pytest` verde localmente; `ruff check .` e `ruff format --check .` sem erros.
- ☐ Workflow do CI com YAML válido, executando lint + testes em 3.11 e 3.12.
- ☐ README com instruções de instalação e uso.
- ☐ **“Verde no CI” só eu posso confirmar.** No relatório, peça que eu faça `git push` e confira a aba Actions. Não faça push.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-01 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-02.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

CARD S1-02 · ADR de arquitetura

2 pts

Objetivo. Registrar em `docs/adr/0001-arquitetura.md` as decisões de arquitetura, as interfaces e as convenções. É o card de maior alavancagem da semana: todo o resto do projeto depende dele. **Este card não produz código em `src/`.**

Passos

1. Crie `docs/adr/0001-arquitetura.md` em português, com as seções: **Status** (comece como “Proposto”), **Contexto**, **Decisão**, **Estrutura de pacotes** (a árvore da seção 5, já com `astrotime.py`), **Convenções** (unidades, tempo, ângulos, quadros, shapes dos arrays, tabela de notação $\psi/\ell/h$), **Interfaces**, **Fluxo de dados**, **Interface gráfica (GUI)**, **Alternativas rejeitadas**, **Simplificações e limitações conhecidas**, **Consequências**.

2. Na seção **Interfaces**, inclua as assinaturas abaixo (blocos de código ilustrativos; ajuste só se encontrar problema real e justifique no relatório).
3. **Fluxo de dados:** Propagator → ECI → (rotação por GMST) → ECEF → geodésico/observador (ENU) → elevação → visibilidade.
4. **Seção GUI (obrigatória e detalhada):** transcreva os requisitos R1-R10 e o layout-alvo (seção 1 deste prompt); inclua o **contrato de dados da cena** (bloco de código abaixo, ajustável com justificativa); e a **decisão de stack** com tabela comparativa de pelo menos 4 alternativas (globe.gl/three.js, CesiumJS, Plotly, PyVista/VTK, Streamlit puro) nos critérios: impacto visual, funcionamento offline, esforço, deploy web, licença/tokens. Recomendação de partida: viewer HTML estático autocontido (globe.gl UMD para o 3D + canvas para o 2D) alimentado por JSON gerado em Python; hospedagem na Semana 6 via Streamlit ou FastAPI. A decisão final é sua: justifique se divergir. Registre também a decisão de vista 3D (Terra fixa/ECEF nesta fase; modo inercial/ECI fica no backlog).
5. **Alternativas rejeitadas** (escreva 1-2 linhas de justificativa cada): lista de objetos Python por instante (lento) versus arrays vetorizados; Skyfield/astropy no runtime versus só na validação; usar M e h da literatura versus ψ e $e\ell l$; arquitetura Walker Delta versus plano único (a SSO exige o mesmo LTAN, logo um plano só).
6. **Simplificações e limitações:** sem precessão/nutação (ECI simplificado); $UT1 \approx UTC$ (erro até $0,9\text{ s} \approx 0,4\text{ km}$ no equador); segundos intercalares ignorados pelo `datetime`; $TEME \neq ECI$ será tratado na Semana 3 (S3-02); o modelo de elementos médios é de primeira ordem em J2 e não serve para a ISS por mais de poucos dias sem TLE fresco.

```

class Propagator(ABC):
    @property
    @abstractmethod
    def epoch(self) -> datetime: ...          # UTC, tz-aware

    @abstractmethod
    def propagate(self, t: ArrayLike) -> tuple[NDArray, NDArray]:
        """t: segundos desde `epoch`, shape () ou (N,).
        Retorna (r, v) em ECI, SI, sempre com shape (N, 3)."""

@dataclass(frozen=True)
class Satellite:
    id: str
    name: str
    propagator: Propagator
    cd: float | None = None          # coeficiente de arrasto
    area_over_mass: float | None = None  # A/m [m²/kg]
    def propagate(self, t): return self.propagator.propagate(t)

class Constellation:
    epoch: datetime          # época comum; converte t para cada satélite
    satellites: Sequence[Satellite]
    def propagate(self, t) -> tuple[NDArray, NDArray]: ...  # cada um com shape (P, N, 3)

@dataclass(frozen=True)
class GroundStation:
    name: str; lat: float; lon: float; alt: float          # rad, rad, m (WGS84)
    @classmethod
    def from_degrees(cls, name, lat_deg, lon_deg, alt_m): ...
    def ecef(self) -> NDArray: ...          # (3,)
    def look_angles(self, r_ecef) -> tuple[NDArray, NDArray, NDArray]: ...  # az, el, alcance

# Contrato "cena" (JSON): Python calcula, o viewer só desenha
{
  "meta": {"epoch_utc": "2026-09-29T12:00:00Z", "duration_s": 21600, "step_s": 10,
    "frame": "ECI simplificado (x -> equinócio vernal)"},
  "earth": {"gmst0_rad": 1.234, "gmst_rate_rad_s": 7.2921158553e-5},
  "satellites": [
    {"id": "ssol", "name": "SSO-1", "color": "#00e5ff",
      "eci_km": [[x, y, z], ...],          # N pontos, um por passo
      "lla": [[lat_deg, lon_deg, alt_km], ...]}  # subponto (aprox. esférica na Semana 1)
  ],
  "stations": [{"id": "sbs", "name": "São Bento do Sapucaí",
    "lat_deg": -22.69, "lon_deg": -45.73, "alt_m": 900}],
  "passes": []          # preenchido a partir da Semana 3
}

```

Critérios de aceite

- [] Documento com todas as seções acima (incluindo GUI, contrato JSON e comparação de stacks) e no máximo ~4 páginas.
- [] Nenhum arquivo em `src/` criado ou alterado neste card.
- [] Seção GUI cobre os requisitos R1-R10, o contrato JSON e a decisão de stack justificada.
- [] Status permanece “Proposto”. **Eu** mudo para “Aceito” após revisar (ou peço que você mude na aprovação).

Notas

- Somente `Propagator` será implementado nesta semana (S1-05). `Satellite`, `Constellation` e `GroundStation` ficam apenas declarados no ADR e serão implementados nas semanas 3-5.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-02 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-03.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

Objetivo. Criar `src/satsim/constants.py` com as constantes físicas e astronômicas, idênticas às dos PDFs, e documentar a armadilha de normalização de `J2`.

Passos

1. Crie `constants.py` com `typing.Final` e docstrings/comentários com unidade e fonte, contendo exatamente as constantes da tabela abaixo.
2. Adicione a constante derivada $\text{OMEGA_SUN} = 2\pi / (\text{T_TROP_DAYS} \cdot 86400)$ (taxa heliossíncrona alvo, Sem4 eq. 26) e os auxiliares `SECONDS_PER_DAY`, `DEG2RAD`, `RAD2DEG`.
3. Escreva no módulo (docstring ou comentário destacado) a **armadilha C20 versus C20**: modelos como o EGM2008 tabulam coeficientes totalmente normalizados, $\text{Cbar20} = -4,8417\text{e-}4$; o fator de normalização é $\text{N20} = \sqrt{5} = 2,2361$, então $\text{C20} = \text{N20} \cdot \text{Cbar20} = -1,0826\text{e-}3$ e $\text{J2} = -\text{C20} = 1,0826\text{e-}3$. Usar `Cbar20` direto no lugar de `C20` erra `J2` por um fator 2,24 e a inclinação heliossíncrona sai completamente fora.
4. Escreva `tests/test_constants.py` (abaixo).

Símbolo no código	Valor	Unidade / observação
MU	3,986004418e14	m³/s² (parâmetro gravitacional da Terra)
R_EARTH	6378,137e3	m (raio equatorial)
OMEGA_EARTH	7,292115e-5	rad/s (rotação da Terra)
J2	1,08262668e-3	adimensional. O texto do PDF usa 1,082627e-3; use o valor do código do PDF
J3	-2,533e-6	adimensional
J4	-1,620e-6	adimensional
T_TROP_DAYS	365,2422	dias (ano trópico). O código do PDF usa 365,24219; a diferença é < 3e-8 relativo

Testes exigidos

- Cada constante igual ao valor da tabela (comparação exata ou `rel=1e-12`).
- `OMEGA_SUN` em graus/dia = 0,985647 (`abs=1e-6`) e em rad/s = 1,991064e-7 (`rel=1e-6`).
- $\text{OMEGA_EARTH} / \text{OMEGA_SUN} \approx 366,2$ (`rel=1e-3`) — a precessão heliossíncrona é ~366 vezes mais lenta que a rotação da Terra.
- $-\text{sqrt}(5) \cdot (-4,8417\text{e-}4)$ reproduz `J2` com `rel=1e-4` (armadilha da normalização).
- `J3 < 0` e `J4 < 0`.

Critérios de aceite

- [] Todos os valores idênticos à tabela.
- [] Testes passando, `ruff` limpo.
- [] Nota sobre C20 vs C20 presente no código.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-03 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-04.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

Objetivo. Criar `src/satsim/elements.py`: resolver a equação de Kepler e converter entre elementos clássicos ($a, e, i, \Omega, \omega, \psi$) e estado (r, v) em ECI, vetorizado.

Passos

1. Defina `@dataclass(frozen=True) KeplerianElements(a, e, i, raan, argp, psi)` (a em m; ângulos em rad; ψ = anomalia média).
2. `solve_kepler(psi, e, tol=1e-14, max_iter=50)`: resolve $E - e \cdot \sin E = \psi$ por Newton-Raphson vetorizado. Normalize ψ para $[-\pi, \pi]$; chute inicial $E_0 = \psi + e \cdot \sin \psi$ (ou π se $e > 0,8$). Só órbitas elípticas: levante `ValueError` se $e < 0$ ou $e \geq 1$.
3. Conversões: `eccentric_to_true`, `true_to_eccentric`, `mean_to_true`, `true_to_mean` (use `arctan2`, sem ambiguidade de quadrante).
4. `elements_to_state(el, mu=MU) → (r, v)`: posição e velocidade no plano perifocal — $r_{pf} = (a(\cos E - e), a\sqrt{1-e^2} \sin E, 0)$ e $v_{pf} = (\sqrt{\mu a}/r) \cdot (-\sin E, \sqrt{1-e^2} \cos E, 0)$ — e rotação 3-1-3 $R_z(\text{raan}) \cdot R_x(i) \cdot R_z(\text{argp})$ para ECI. `psi` pode ser array; devolva shapes $(N,3)$.
5. `state_to_elements(r, v, mu=MU) → KeplerianElements`: use $\ell = r \times v$; vetor dos nós $n = z_{\text{hat}} \times \ell$; vetor excentricidade $e_{\text{vec}} = ((v^2 - \mu/r) r - (r \cdot v) v)/\mu$; a pela energia; $i = \arccos(\ell_z/|\ell|)$; Ω, ω e v por `arctan2` com os sinais corretos ($\omega < \pi$ se a componente z do vetor excentricidade for > 0 ; $v < \pi$ se $r \cdot v > 0$), depois ψ a partir de v .
6. **Casos degenerados (documente na docstring, citando Sem4 §3.3)**: se $e < 1e-11$ (circular), defina `argp = 0` e ponha todo o ângulo em v , que passa a ser o argumento de latitude u ; se $i < 1e-11$ ou $i > \pi - 1e-11$ (equatorial), defina `raan = 0` com a linha dos nós no eixo x ; se ambos, use a longitude verdadeira. Exponha `ECC_TOL` e `INC_TOL` como constantes do módulo. A singularidade é da carta de coordenadas, não da física.
7. Escreva `tests/test_elements.py` (abaixo).

Testes exigidos

- **Equação de Kepler**: para $e \in \{0; 1e-3; 0,1; 0,7; 0,95\}$ e 1000 valores de ψ em $[-\pi, \pi]$, $|E - e \sin E - \psi| < 1e-12$.
- **Órbita de projeto**: $a = 6888,089e3$, $e = 1,074e-3$, $i = 97,4396^\circ$, $\text{raan} = 30^\circ$, $\text{argp} = 90^\circ$, $\text{psi} = 45^\circ$. Elementos → estado → elementos: a com erro $< 1e-6$ m; ângulos $< 1e-9$ rad. Estado → elementos → estado: posição $< 1e-6$ m e velocidade $< 1e-9$ m/s.
- **Degenerados (round-trip pelo estado)**: circular inclinada ($e = 0$), equatorial excêntrica ($i = 0$), circular equatorial ($e = 0$ e $i = 0$), e o caso retrógrado equatorial ($i = \pi$).
- **Invariantes**: $a(1-e) \leq |r| \leq a(1+e)$; energia $v^2/2 - \mu/r = -\mu/(2a)$ com $\text{rel}=1e-12$; $|\ell| = \sqrt{\mu \cdot a \cdot (1-e^2)}$; $\ell \cdot r = 0$; $i > 90^\circ$ implica $\ell_z < 0$ (retrógrada).
- **Eq. 12 da Sem4**: $z/r = \sin i \cdot \sin u$ com $u = \text{argp} + v$, e a latitude máxima ao longo de uma órbita completa = $82,56^\circ$ ($\text{abs}=0,01^\circ$) para $i = 97,4396^\circ$.
- **Vetorização**: `psi` com shape $(N,)$ gera r e v com shape $(N,3)$, iguais ao cálculo escalar ponto a ponto.

Critérios de aceite

- [] Ida e volta (elementos → estado → elementos, e estado → elementos → estado) com erro de posição $< 1e-6$ m.
- [] Kepler resolvido com resíduo $< 1e-12$ até $e = 0,95$.
- [] Casos $e \rightarrow 0$ e $i \rightarrow 0$ tratados e documentados.
- [] Vetorização funcionando para `psi` array; `ruff` limpo.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-04 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-05.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

CARD S1-05 · KeplerPropagator vetorizado

3 pts

Objetivo. Implementar a interface base `Propagator` (ABC) e o propagador kepleriano puro (sem perturbações).

Passos

1. Crie `src/satsim/propagators/__init__.py` (exporte `Propagator` e `KeplerPropagator`) e `base.py` com a ABC exatamente como no ADR (epoch e `propagate`), mais um helper `as_time_array(t)` que converte escalar ou sequência em `ndarray` float 1D de shape `(N,)`.
2. Crie `kepler.py` com `KeplerPropagator(elements: KeplerianElements, epoch: datetime, mu=MU).epoch` deve ser tz-aware (levante `ValueError` caso contrário). Movimento médio $n = \sqrt{\mu/a^3}$; $\psi(t) = \psi_0 + n \cdot t$.
3. `propagate(t)` chama `elements_to_state` vetorizado e devolve `(r, v)` com shape `(N, 3)`, mesmo para `t` escalar.
4. Exponha as propriedades `mean_motion` [rad/s] e `period` ($T_{\text{kep}} = 2\pi/n$) [s].
5. Escreva `tests/test_kepler_propagator.py` (abaixo). Para a validação cruzada, integre a equação de dois corpos com `scipy.integrate.solve_ivp` (D0P853, `rtol=1e-13`, `atol=1e-6`) por 1 dia e compare com o analítico.

Testes exigidos

- Conservação em 30 dias (energia, $|l|$, direção de l).
- Período: `KeplerPropagator(...).period` $\approx 5689,30$ s para o semieixo de projeto.
- Retorno ao estado inicial após `t = period`.
- Órbita circular ($e = 0$) com $a = 6888,089$ km: velocidade ≈ 7607 m/s ($\text{abs}=1$ m/s).
- `propagate([0.0])` igual a `elements_to_state(elementos)`.
- Latitude máxima do subponto aproximado ($\arcsin(z/r)$) ao longo de uma órbita $\approx 82,56^\circ$ ($\text{abs}=0,01^\circ$).
- Validação cruzada com `solve_ivp` (1 dia, erro < 1 m).
- Época sem timezone levanta `ValueError`.

CrITÉrios de aceite

- [] Energia específica e $|l|$ conservados a $1e-10$ relativo em 30 dias (amostragem de 60 s); direção de l constante.
- [] $T_{\text{kep}}(a = 6888,089 \text{ km}) = 5689,30 \text{ s}$ ($\text{abs}=0,05 \text{ s}$). O “510 km” é o rótulo nominal de altitude; o semieixo exato é 6888,089 km ($h = 509,952 \text{ km}$).
- [] Depois de exatamente um período, o estado retorna ao inicial (posição $< 1e-4$ m).
- [] Diferença para a integração numérica de 1 dia < 1 m.
- [] Toda a API funciona com `t` escalar e com `t` array; `ruff` limpo.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-05 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-06.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

Objetivo. Criar `src/satsim/astrotime.py` com conversões entre `datetime` UTC, Data Juliana e tempo sideral médio de Greenwich (ΘG).

Passos

1. `datetime_to_jd(dt)` → `float`: exige `datetime` tz-aware (levante `ValueError` se for naive), converte para UTC e devolve a Data Juliana. Sugestão simples e exata: $JD = 2440587,5 + \text{timestamp}/86400$.
2. `jd_to_datetime(jd)` → `datetime` UTC tz-aware (arredonde para microssegundos).
3. `jd_from_epoch(epoch_jd, t)` → `jd`: $\text{epoch_jd} + t/86400$, vetorizado sobre `t` (segundos desde a época).
4. `gmst(jd_ut1)` → `rad` em $[0, 2\pi)$, vetorizado, pela fórmula IAU-82 de Vallado (eq. 3-47): $\theta [s] = 67310,54841 + (876600 \cdot 3600 + 8640184,812866) \cdot T + 0,093104 \cdot T^2 - 6,2e-6 \cdot T^3$, com $T = (JD - 2451545,0)/36525$; converta segundos de tempo para radianos ($\theta \bmod 86400 \cdot 2\pi/86400$).
5. Documente no módulo: (a) UTC é tratado como UT1 ($|UT1-UTC| < 0,9 \text{ s}$, $\approx 0,4 \text{ km}$ no equador); (b) `datetime` ignora segundos intercalares; (c) a taxa do GMST ($7,2921158553e-5 \text{ rad/s}$) difere levemente da constante `OMEGA_EARTH` usada nos documentos ($\approx 0,15''/\text{dia}$, $\approx 5 \text{ m}/\text{dia}$ no equador) — aqui vale a fórmula completa do GMST.
6. Escreva `tests/test_astrotime.py` (abaixo).

Testes exigidos

- `JD(2000-01-01 12:00 UTC) = 2451545,0`; `JD(1970-01-01 00:00 UTC) = 2440587,5`; `JD(2000-01-01 00:00 UTC) = 2451544,5`.
- `Datetime` em outro fuso (ex.: `-03:00`) é convertido corretamente para UTC; naive levanta `ValueError`.
- `gmst(2451545,0) = 280,46061837°` ($\text{abs}=1e-6^\circ$).
- Vallado, Exemplo 3-5 (20/08/1992, 12:14:00 UT1): `JD = 2448855,009722` e `GMST  $\approx 152,5788^\circ$` ($\text{abs}=1e-3^\circ$). Se divergir, confie no Skyfield e reporte a discrepância.
- Taxa do GMST: `gmst(jd+1) - gmst(jd)` (módulo 2π) $\approx 0,98564736629^\circ/\text{dia}$ ($\text{abs}=1e-6^\circ$) — o excesso sobre 360° é o movimento aparente do Sol, o mesmo número da condição heliossíncrona.
- **Skyfield (validação):** `pytest.importorskip("skyfield"); ts = load.timescale(builtin=True)`; para cada JD use `ts.ut1_jd(jd).gmst` (em horas; converta para rad), tratando UTC como UT1 para comparar como o mesmo critério. Diferença angular (com wrap em 2π) convertida para segundos de tempo ($\Delta\theta \cdot 86400/2\pi$) $< 0,1 \text{ s}$.
- Vetorização: `gmst` sobre array de JDs devolve array com o mesmo shape.

Critérios de aceite

- [] GMST confere com Skyfield a $< 0,1 \text{ s}$ (de tempo sideral) em 200 datas aleatórias entre 1990 e 2040.
- [] Round-trip `datetime` ↔ JD com erro $< 100 \mu\text{s}$.
- [] Funções vetorizadas onde indicado; naive `datetime` rejeitado; `ruff` limpo.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-06 CONCLUÍDO — aguardando sua aprovação para iniciar o S1-07.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

Objetivo. Reunir, em um único arquivo versionado, todos os números de referência dos PDFs (Apêndice A), com a fonte de cada um, para reuso nas próximas semanas; e testar a consistência interna dos números para pegar erros de transcrição.

Passos

1. Crie `tests/golden/__init__.py` e `tests/golden/reference_values.py` com estruturas simples (dicionários ou dataclasses congeladas) contendo os valores do **Apêndice A**. Cada grupo deve ter um comentário com a fonte (documento, seção, tabela ou equação). Valores em SI e graus, como aparecem nos PDFs; deixe explícito o sufixo da unidade no nome (`h_km`, `i_deg`, `tnod_s`).
2. Crie `tests/test_golden_consistency.py` com testes de consistência interna (abaixo). Nenhum desses testes depende de código de semanas futuras, exceto os dois primeiros que usam o que já existe.
3. **Não tente testar agora** valores que dependem de módulos ainda inexistentes (solver de SSO, J2 médio, constelação). Eles ficam apenas registrados. Cada card das semanas seguintes importará este arquivo.
4. Adicione no README uma seção curta “Golden tests” explicando o propósito do arquivo e como adicionar novos valores.

Testes exigidos

- **Com código existente:** `constants.J2` igual ao `golden`; `KeplerPropagator` com `a = 6888,089 km` tem `period ≈ golden tkep_s` (`abs=0,05 s`).
- **Tabela 1 (Sem4):** para cada linha, `86400 / tnod_s ≈ rev_por_dia` (`abs=0,001`).
- **Tabela 3 (Sem4):** para cada linha com `gcd = 1`: `gcd(N, D) == 1` (usando `math.gcd`); `360/N ≈ grade_deg` (`abs=0,001`); `2π·R_EARTH/N ≈ grade_km` (`abs=0,1`). Linhas degeneradas (93/6 e 90/6) têm `gcd > 1`.
- **Órbita de projeto 91/6:** `86400·D/N ≈ tnod_s` (5696,70 s, `abs=0,01`); `N/D ≈ 15,167`; `-360·D/N ≈ -23,736°` (`abs=0,001`); `h = a - R_EARTH ≈ 509,952 km`.
- **Consistência entre grupos:** `tnod - tkep ≈ 7,4 s`; `OMEGA_SUN` em `°/dia = 0,985647`; latitude máxima `180° - i = 82,56°`.
- Nenhum valor duplicado com valores diferentes entre grupos (ex.: `h` de projeto igual em todos).

Critérios de aceite

- [] Arquivo `reference_values.py` versionado, importado pelos testes, com fonte comentada em cada grupo.
- [] Testes de consistência interna passando (detectam typos nas tabelas).
- [] `ruff` e `pytest` verdes (suíte completa da semana).

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-07 CONCLUÍDO – aguardando sua aprovação para iniciar o S1-08.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

Objetivo. Entregar a **primeira versão visível do produto**: uma página que mostra, na mesma tela e sincronizados, um **globo 3D** com satélites em movimento e o **mapa 2D com o traço no solo** (requisitos R1-R10 da seção 1). Alimentada pelo `KeplerPropagator`. É um spike: prioridade é impacto visual e a arquitetura correta (Python calcula, viewer desenha), não funcionalidades de missão.

Passos

1. Pré-requisitos: S1-05 e S1-06 aprovados. Leia a seção GUI do ADR e siga-a; se algo estiver inviável, justifique o desvio no relatório.
2. Crie `src/satsim/app/__init__.py` e `scene_export.py` com `build_scene(satellites, stations, epoch, duration_s, step_s) → dict` no formato do contrato do ADR. Como `Satellite` só existe na Semana 5, defina uma `@dataclass SceneSatellite(id, name, color, propagator)` **provisória** (documento que será substituída na S5-01). Cálculo: $r_{eci} = R_z(gmst(t)) \cdot r_{eci}$; $lat = \arcsin(z/r)$, $lon = \arctan2(y, x)$ normalizada em $[-180^\circ, 180^\circ]$, $alt = r - R_{EARTH}$ (aproximação esférica, documentada: o geodésico WGS84 vem na S2-03). Saída em km e graus, arredondada a ~4 casas para reduzir o tamanho do JSON.
3. Baixe e **versione** os assets do viewer: textura equirretangular da Terra de domínio público (ex.: NASA Blue Marble, 2048x1024, ≤ ~2 MB) em `static/textures/` e o bundle UMD do `globe.gl` (versão fixada) em `static/vendor/`. Registre fonte, versão e licença de cada um em `docs/creditos.md`. Se não houver acesso à internet, pare e peça que eu coloque os arquivos.
4. Crie o viewer em `src/satsim/app/static/` (`viewer.html`, `viewer.js`, `viewer.css`) com **duas áreas na mesma tela** (layout-alvo da seção 1): **3D**: Terra texturizada, fundo com estrelas, brilho atmosférico, câmera arrastável com zoom, satélites como marcadores luminosos com rótulo e cor própria, **trilha recente** e **órbita prevista**, marcador da estação de solo. **2D**: canvas desenhando a mesma textura em projeção equirretangular, com grade de lat/lon, subponto atual, **traço passado (linha cheia) e futuro (tracejada)**, **quebra correta no antimeridiano** e marcador da estação. **Controles**: play/pause, velocidade (1x, 10x, 60x, 300x, 1000x), linha do tempo arrastável, relógio UTC, botão “início”, checkbox por satélite, legenda.
5. Regras do viewer: uma única variável `simTime` comanda 3D e 2D; loop com `requestAnimationFrame`; interpolação temporal entre amostras que trate o salto de $\pm 180^\circ$ na longitude (interpole em 3D cartesiano ou desenrole a longitude); **nenhuma equação orbital em JS**; ler qualquer número de satélites do JSON (R7).
6. Crie `scripts/build_demo.py`: monta a cena (SSO de projeto: $a = 6888,089$ km, $e = 1,074e-3$, $i = 97,4396^\circ$, $raan = 30^\circ$, $argp = 90^\circ$; mais um segundo objeto kepleriano com $i = 51,64^\circ$ e $a \approx 6798$ km só para provar o multi-satélite — rotule-o como “exemplo kepleriano”, **não** é a ISS real, que entra na Semana 3; janela de 6 h, passo 10 s; estação São Bento do Sapucaí) e gera `demo/index.html` **autocontido**: JSON embutido em `<script>`, JS/CSS inline ou relativo, textura como data URI. Deve abrir com **duplo clique (file://)**, **sem servidor e sem internet**. Use scripts clássicos, não módulos ES. Adicione `demo/` ao `.gitignore`.
7. Escreva `tests/test_scene_export.py` (abaixo). Se node estiver disponível, valide a sintaxe do JS com `node --check`. Se conseguir gerar uma captura de tela (ex.: Playwright, **sem adicioná-lo às dependências do projeto**), salve `docs/img/demo.png`; senão, pule e diga no relatório.
8. Atualize o README com a seção “Demo visual”: como gerar (`python scripts/build_demo.py`), como abrir, como adicionar um satélite à demo (uma linha) e os controles.

Testes exigidos

- Estrutura do JSON: chaves `meta`, `earth`, `satellites`, `stations`, `passes`; comprimento dos arrays = `duration_s/step_s + 1`; JSON serializável (`json.dumps`).
- SSO de projeto: $|lat| \leq 82,56^\circ + 0,05^\circ$ e a latitude máxima $\approx 82,56^\circ$ ($abs=0,1^\circ$) em 6 h; $lon \in [-180^\circ, 180^\circ]$.
- $|eci_km|$ entre $a(1-e)$ e $a(1+e)$ (em km); $alt_km \approx |eci| - R_{EARTH}$.

- Consistência: `lla` recalculado a partir de `eci_km` e `gmst` concorda com o exportado (`abs=1e-6°`).
- Multi-satélite: a mesma função exporta 1, 2 e 3 satélites sem alteração de código.
- Continuidade: nenhum salto de longitude entre amostras consecutivas $> 5^\circ$ **exceto** o de $\pm 360^\circ$ no antimeridiano (contado e verificado).

Critérios de aceite

- [] `python scripts/build_demo.py` gera `demo/index.html`; ao abri-lo com duplo clique aparecem, **na mesma tela**, o globo 3D com ≥ 2 satélites em movimento e o mapa 2D com subpontos e traço passado/futuro, sincronizados.
- [] Controles funcionando: play/pause, velocidade, linha do tempo, relógio UTC, início, mostrar/ocultar satélite.
- [] Estação de São Bento do Sapucaí marcada nos dois painéis.
- [] Traço 2D sem linhas horizontais atravessando o mapa no antimeridiano.
- [] Adicionar um terceiro satélite exige alterar **uma linha** em `build_demo.py` (demonstre no relatório).
- [] Funciona offline: nenhum request externo (verifique com `grep -n "http"` nos assets, ignorando comentários de licença, e descreva o resultado).
- [] Estética: tema escuro, estrelas, atmosfera, animação fluida. Descreva no relatório o que foi feito e o desempenho percebido; **eu avalio visualmente e posso pedir iterações dentro deste card**.
- [] Testes de export passando; `ruff` limpo; créditos e licenças dos assets registrados.

Notas

- **Fora do escopo:** passagens, visibilidade, eclipse, SGP4/TLE, serviço web, deploy, J2 e arrasto. Não antecipe as semanas seguintes.
- **Vista 3D:** nesta fase, Terra fixa (ECEF) usando `lla`. Modo inercial (Terra girando sob órbitas fixas) fica no backlog; `eci_km` já vai no contrato.
- A qualidade visual é subjetiva: não declare “concluído” sem descrever o que o usuário verá ao abrir o arquivo.

PARE AQUI. Rode lint e testes, faça o commit do card, envie o relatório de encerramento (seção 3) e termine sua resposta com esta linha exata:

CARD S1-08 CONCLUÍDO – aguardando sua aprovação para fechar a Semana 1.

Não inicie o próximo card enquanto eu não escrever “aprovado” ou “próximo”.

7. Fechamento da Semana 1

Depois que eu aprovar o S1-08, faça **um relatório de fechamento** e pare (não comece a Semana 2):

- Resumo do que foi entregue e status dos 8 cards (com os commits).
- Saída completa de `pytest --cov=satsim` e de `ruff check ..`
- Dívida técnica ou pontos frágeis que você notou (ex.: decisões que a Semana 2 pode precisar rever).
- Sugestões de ajustes no ADR (a ser marcado como “Aceito” por mim).
- Lembrete: `git push` da branch `semana-1` e abertura do PR é comigo.

Apêndice A — Valores de referência (para o S1-07)

Fonte: PDFs “Como se comporta um satélite em órbita heliossíncrona?” (Semana 4, **Sem4**) e “Por que satélites em LEO caem?” (Semana 3, **Sem3**). Os valores da Sem4 são de teoria secular de primeira ordem em J2; use-os como valores de projeto preliminar.

A.1 Constantes

Grandeza	Valor	Fonte
μ	$3,986004418e14 \text{ m}^3/\text{s}^2$	Sem4, tabela de símbolos
$R_\oplus$	6378,137 km	Sem4, tabela de símbolos
$\omega_\oplus$	$7,292115e-5 \text{ rad/s}$	Sem4, tabela de símbolos
$J_2 \cdot J_3 \cdot J_4$	$1,08262668e-3 \cdot -2,533e-6 \cdot -1,620e-6$	Sem4 §2.2 e código §9.1
T_{trop}	365,2422 dias	Sem4, símbolos
Taxa heliossíncrona	$0,985647^\circ/\text{dia} = 1,991064e-7 \text{ rad/s}$	Sem4 eq. 26
$C_{\text{bar20}} \cdot N_{20} \cdot C_{20}$	$-4,8417e-4 \cdot \sqrt{5} = 2,2361 \cdot -1,0826e-3$	Sem4 §2.1 (armadilha)
Clairaut: J2 hidrostático $\cdot m$	$1,0814e-3 \cdot 3,4614e-3$	Sem4 §2.2
$T_{\text{dia,nod}}$ em SSO	86400,01 s	Sem4 §7.2

A.2 Órbita de projeto 91/6 (h nominal 510 km)

Grandeza	Valor	Grandeza	Valor
N / D	91 / 6 (gcd = 1)	a	6888,089 km
h (nominal)	509,952 km (509,95)	i	$97,4396^\circ$
e (congelada)	$1,074e-3$	ω (congelada)	90°
T_{kep}	5689,30 s	T_{nod}	$5696,70 \text{ s} = 94,945 \text{ min}$
$T_{\text{nod}} - T_{\text{kep}}$	7,4 s (0,13%)	v (circular)	7607 m/s
Ω -ponto	$+0,985647^\circ/\text{dia}$	ω -ponto (só J2)	$-3,487^\circ/\text{dia}$ (líquido 0)
$\Delta\lambda$ por revolução	$-23,736^\circ$	Δgrade	$3,956^\circ = 440,4 \text{ km}$
rev/dia	15,167	Altitude instantânea	502,5 a 517,4 km (excursão 14,8 km)
Latitude máxima	$82,56^\circ$ ($180^\circ - i$)	β^* (sem eclipse)	$67,8^\circ$ (510 km)
Eclipse (LTDN 10h30)	$\sim 35 \text{ min}/\text{órbita}$ ($\sim 37\%$)	β nessa órbita	-24° a -17°

A.3 Tabela 1 (Sem4): SSO circulares

h [km]	i [°]	T_{nod} [s]	rev/dia	Missão de referência
300	96,672	5438,8	15,886	
400	97,030	5561,1	15,536	
500	97,402	5684,4	15,200	SkySat, PlanetScope
510	97,440	5696,7	15,167	órbita de projeto
600	97,788	5808,5	14,875	
705	98,208	5939,8	14,546	Landsat 8/9
786	98,544	6041,8	14,300	Sentinel-2
800	98,603	6059,5	14,259	
1000	99,479	6313,9	13,684	
1200	100,420	6571,9	13,147	

A.4 Tabela 3 (Sem4): SSO de repetição

N	D	gcd	h [km]	i [°]	Δgrade [°]	no equador [km]
95	6	1	314,80	96,724	3,789	421,8

N	D	gcd	h [km]	i [°]	Δ grade [°]	no equador [km]
93	6	3	410,63	97,069	—	degenerada (ciclo real 2 dias)
91	6	1	509,95	97,440	3,956	440,4
106	7	1	517,19	97,467	3,396	378,1
121	8	1	522,62	97,488	2,975	331,2
90	6	6	560,99	97,636	—	degenerada (ciclo real 1 dia)
89	6	1	612,99	97,839	4,045	450,3
175	12	1	692,83	98,159	2,057	229,0
233	16	1	699,59	98,186	1,545	172,0 (Landsat)
143	10	1	786,12	98,545	2,517	280,2 (Sentinel-2)

A.5 Taxas seculares de J2 (Sem4 §3.5)

Órbita	i	Ω -ponto [°/dia]	ω -ponto [°/dia]
Molniya (inclinação crítica)	63,435°	−0,148	0
Polar exata	90°	0	−3,806
SSO de projeto	97,44°	+0,986	−3,487 (só J2)
ISS (h = 420 km)	51,64°	−4,947	+3,690

- ISS: precessão do plano de $\approx -5^\circ/\text{dia}$, volta completa a cada ≈ 72 dias (Sem4 §3.4).
- Teto de altitude da SSO: $a \leq 12352$ km ($h \leq 5974$ km), com $i = 180^\circ$ (Sem4 §4.3).
- Sensibilidade (Sem4 §6.1, eq. 32): $\tan i = -7,66$ na órbita de projeto; $\delta h = +1$ km $\Rightarrow -0,73$ min/ano de deriva de LTAN; $\delta i = +0,01^\circ \Rightarrow +1,9$ min/ano.
- Deriva da grade (Sem4 eq. 40): $\delta a = -100$ m $\Rightarrow 0,047^\circ$ por ciclo $\approx 5,2$ km no equador ($D = 6$).
- Cadência de manobras (Sem4 eq. 41, $\dot{a} = -35$ m/dia): banda ± 1 km $\rightarrow 2,6$ dias; ± 5 km $\rightarrow 5,7$ dias; ± 20 km $\rightarrow 11,4$ dias.
- Sem3 (arrasto), 400 km, $CD = 2,2$, $A/m = 0,005$ m²/kg; $\rho = 3,7e-12$ kg/m³; $aD \approx 1,2e-6$ m/s²; decaimento ≈ 184 m/dia; tempo de vida ≈ 1 ano. A 500 km: $\rho = 7,0e-13$; $\dot{a} \approx -35$ m/dia; tempo de vida ≈ 5 anos.

A.6 Constelação de 6 satélites (Sem4 Tabela 4)

Satélite	ψ_0 [°]	Atraso no nó [s]
S1	0	0
S2	300	949,5
S3	240	1898,9
S4	180	2848,4
S5	120	3797,8
S6	60	4747,3

Todos compartilham a , e , i , Ω e ω . Verificação da grade (Sem4 §9.2): $P = 1$ em 6 dias e $P = 6$ em 1 dia $\rightarrow 91$ nós, passo $3,9560^\circ$; $P = 7$ em 1 dia $\rightarrow 107$ nós, passo de $0,5651^\circ$ a $3,3909^\circ$ (não uniforme).